SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Name:

Sulfri Trainer Portal (Personal Brand Edition)

Version: 1.0 Prepared For: Personal Deployment Prepared By: System Specification Draft

---

1. INTRODUCTION

1.1 Purpose

This document defines the functional and non-functional requirements for the Sulfri Trainer Portal web application. The system will serve as a personal trainer management platform consisting of:

1. Public profile website
2. Student registration via class join links
3. Private backoffice (admin control panel)

This document is intended for developers and AI agents responsible for implementation.

---

2. SYSTEM OVERVIEW

The system consists of two main components:

2.1 Public Frontend

Accessible by anyone. - View trainer profile - View teaching experience - Download profile documents (PDF) - Register for classes via join link

2.2 Admin Backoffice

Accessible only via authentication. - Manage classes - View and export student registrations - Manage profile information - Upload downloadable files - Control visibility of classes on public profile

---

3. SYSTEM ARCHITECTURE

Recommended Architecture: - Frontend: Next.js (App Router) - Backend: Next.js API Routes - Database: PostgreSQL - Authentication: JWT or NextAuth - File Storage: Cloud object storage (e.g., Supabase Storage / S3-compatible) - Hosting: Cloud platform (e.g., Vercel)

Architecture Type: Monolithic web application with API layer.

---

4. USER ROLES

4.1 Admin

- Full system access
- Manage all content

4.2 Public User

- View profile
- Register for classes
- Download public documents

---

5. FUNCTIONAL REQUIREMENTS

5.1 AUTHENTICATION MODULE

FR-1: Admin must log in using email and password. FR-2: Passwords must be hashed using secure hashing algorithm. FR-3: Admin routes must be protected. FR-4: Unauthorized access must redirect to login page.

5.2 CLASS MANAGEMENT MODULE

FR-5: Admin must be able to create a class. FR-6: System must auto-generate a unique join_code for each class. FR-7: Admin must be able to edit class details. FR-8: Admin must be able to delete class. FR-9: Admin must be able to toggle join_enabled. FR-10: Admin must be able to toggle show_on_public_profile. FR-11: Admin must be able to mark class status (draft/published/completed/cancelled).

Class Fields: - Title - Client Name - Client Type - Topic Category - Mode (online/physical/hybrid) - Location - Start DateTime - End DateTime - Notes - Status - Join Code - Join Enabled (boolean) - Show on Public Profile (boolean)

5.3 STUDENT REGISTRATION MODULE

FR-12: Public user must access registration form via /join/{join_code}. FR-13: System must validate join_code. FR-14: If join_enabled = false, registration must be blocked. FR-15: Registration form must include: - Full Name (required) - Email (required) - Phone (optional) - Organization (optional) - Consent Checkbox (required) FR-16: Upon submission, system must store registration linked to class. FR-17:

System must display confirmation page. FR-18: System must prevent duplicate registration using same email for same class.

5.4 REGISTRATION MANAGEMENT

FR-19: Admin must view list of registrations per class. FR-20: Admin must search registrations by name or email. FR-21: Admin must export registrations to CSV format. FR-22: Export must include timestamp of registration.

5.5 PUBLIC PROFILE MODULE

FR-23: Public profile page must display: - Display Name - Headline - Biography - Core Expertise - Key Statistics (total classes, years, etc.) FR-24: Teaching experience page must list classes where: - status = completed - show_on_public_profile = true FR-25: Experience page must allow filtering by year and topic category. FR-26: Profile page must display last_updated_at.

5.6 DOWNLOADS MODULE

FR-27: Admin must upload PDF documents. FR-28: Admin must toggle document visibility (public/private). FR-29: Public downloads page must show only public files. FR-30: Public users must be able to download files.

---

6. DATABASE DESIGN

6.1 admin_users

- id (PK)
- name
- email (unique)
- password_hash
- role
- created_at

6.2 classes

- id (PK)
- title
- client_name
- client_type
- topic_category
- mode
- location

- start_datetime
- end_datetime
- notes
- status
- join_code (unique)
- join_enabled (boolean)
- show_on_public_profile (boolean)
- created_at
- updated_at

6.3 registrations

- id (PK)
- class_id (FK)
- full_name
- email
- phone
- organization
- consent_flag
- registered_at

6.4 downloads

- id (PK)
- title
- file_url
- is_public (boolean)
- updated_at

6.5 profile_settings

- id (single row)
- display_name
- headline
- bio
- email
- phone
- linkedin_url
- location_base
- profile_photo_url
- last_updated_at

---

7. API ENDPOINTS

Authentication: POST /api/auth/login POST /api/auth/logout

Classes: GET /api/classes POST /api/classes PUT /api/classes/{id} DELETE /api/classes/{id}

Registration: GET /api/join/{join_code} POST /api/join/{join_code}

Registrations: GET /api/classes/{id}/registrations GET /api/classes/{id}/export

Profile: GET /api/profile PUT /api/profile

Downloads: GET /api/downloads POST /api/downloads DELETE /api/downloads/{id}

---

8. NON-FUNCTIONAL REQUIREMENTS

8.1 Security

- HTTPS required
- Password hashing (bcrypt or equivalent)
- Input validation
- Rate limiting on registration endpoint
- Basic spam prevention

8.2 Performance

- Page load under 2 seconds
- Support minimum 500 concurrent users

8.3 Scalability

- Stateless backend
- Database indexing on join_code and email

8.4 Compliance

- Include consent checkbox for data usage
- Allow deletion of registration upon request

---

9. MVP SCOPE

Included: - Admin login - Class CRUD - Join link registration - Registration export CSV - Public profile page - Public downloads page

Excluded (Future Phase): - Student login portal - Automated email notifications - Payment integration - Certificate auto-generation

---

10. ACCEPTANCE CRITERIA

1. Admin can create class and receive join link.
2. Public user can register successfully.
3. Admin can export registration list.
4. Completed classes appear automatically on public experience page.
5. Public profile updates dynamically.
6. Downloads are accessible publicly.

END OF DOCUMENT